Recursion — Complete Guide

Enhanced Beginner Edition — original content preserved, with added plain-English explanations, brute-force reasoning, and "why we optimize" insight for every problem.

126. Recursive Implementation of Atoi

Understanding the Problem NEW

This is the same Atoi conversion from the Strings chapter (problem 90), but now implemented recursively instead of with an explicit loop — process one character, then recursively handle 'the rest of the string,' carrying along whatever state (sign, accumulated number so far) is needed between calls.

Brute Force Approach NEW

Nothing new to contrast conceptually here — the actual parsing logic (skip whitespace, handle sign, accumulate digits, watch for overflow) is identical to problem 90. The only difference is implementation style: iterative loop vs recursive calls.

Why We Optimize NEW

Recursion is a genuinely useful practice exercise here because it forces you to think about what **state must be threaded through** each recursive call (the running number built so far, the sign, whether you've started reading digits yet) — since there's no shared loop variable to fall back on, every recursive call must explicitly receive and pass along everything it needs. This is good preparation for later problems in this chapter, where recursive state-passing becomes essential (like backtracking problems further down).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Convert string to integer using recursion.

Visual Diagram

```
Input: " -42"
       Output: -42

       Recursive steps:
       atoi(" -42", 0)
         → skip spaces: atoi("-42", 1)
           → handle sign: atoi("42", 2, sign=-1)
             → parse digits: atoi("2", 3, result=4)
```

```
            → parse digits: atoi("", 4, result=42)
              → return 42 * sign = -42

    Input: "4193 with words"
    atoi("4193 with words", 0)
      → parse digits: atoi("193 with words", 1, result=4)
        → parse digits: atoi("93 with words", 2, result=41)
          → parse digits: atoi("3 with words", 3, result=419)
            → parse digits: atoi(" with words", 4, result=4193)
              → stop at space, return 4193
```

Solution Approach

Concept: Recursively process characters with state tracking.

Algorithm

```
Algorithm ATOI(str, index, sign, result):
          If index >= n:
              Return result * sign

          If str[index] == ' ':
              Return ATOI(str, index+1, sign, result)
          Else If str[index] == '+' or str[index] == '-':
              If result == 0:
                  sign = str[index] == '-' ? -1 : 1
                  Return ATOI(str, index+1, sign, result)
          Else If str[index] >= '0' and str[index] <= '9':
              digit = str[index] - '0'
              If result > (Integer.MAX_VALUE - digit) / 10:
                  Return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE
              result = result * 10 + digit
              Return ATOI(str, index+1, sign, result)
          Else:
              Return result * sign

          Return result * sign
```

Java Implementation

```
class Solution {
          public int myAtoi(String s) {
              return atoi(s, 0, 1, 0);
          }

          private int atoi(String s, int index, int sign, int result) {
              if (index >= s.length()) {
                  return result * sign;
              }
```

```
                char c = s.charAt(index);

                // Skip spaces
                if (c == ' ') {

                    return atoi(s, index + 1, sign, result);
                }

                // Handle sign
                if ((c == '+' || c == '-') && result == 0) {
                    sign = (c == '-') ? -1 : 1;
                    return atoi(s, index + 1, sign, result);
                }

                // Parse digits
                if (c >= '0' && c <= '9') {
                    int digit = c - '0';
                    // Check overflow
                    if (result > (Integer.MAX_VALUE - digit) / 10) {
                        return sign == 1 ? Integer.MAX_VALUE : Integer.MIN_VALUE;
                    }
                    result = result * 10 + digit;
                    return atoi(s, index + 1, sign, result);
                }

                // Any other character stops parsing
                return result * sign;
            }
        }
```

Time & Space Complexity

Time Complexity: O(n) | Space Complexity: O(n) recursion stack

127. Pow(x, n)

Understanding the Problem NEW

Compute x raised to the power n (x^n) — e.g. $2^5 = 512$. The obvious method of multiplying x by itself n times works, but the recursive version here is designed to be dramatically faster for large n.

Brute Force Approach NEW

The direct approach: multiply x by itself n times in a loop (or n recursive calls, one multiplication each) — O(n) multiplications for x^n.

Why We Optimize NEW

The key insight: **$x^n = (x^{n/2})^2$** when n is even, and $x \times x^{n-1}$ when n is odd. This means you can compute x^n by first computing $x^{n/2}$ recursively (a problem *half the size*) and then just squaring that one result — instead of n multiplications, you only need about $\log_2 n$ of them, since each recursive call halves the exponent. This 'fast exponentiation' trick (also called exponentiation by squaring) turns O(n) into O(log n), and the same halving-the-problem idea is exactly what made Merge Sort and Binary Search fast earlier in this book.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Calculate x raised to power n using recursion.

Visual Diagram

```
pow(2, 10):
         → pow(2, 5) * pow(2, 5)
           → pow(2, 2) * 2 * pow(2, 2) * 2
             → pow(2, 1) * pow(2, 1) * 2 * pow(2, 1) * pow(2, 1) * 2
               → 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 * 2 = 1024

       Optimized:
       pow(2, 10):
         temp = pow(2, 5) = 32
         return temp * temp = 1024

       pow(2, 5):
```

```
          temp = pow(2, 2) = 4
          return temp * temp * 2 = 32

        pow(2, 2):
          temp = pow(2, 1) = 2
          return temp * temp = 4
```

Solution Approach

Concept: Divide and conquer with fast exponentiation.

Algorithm

```
Algorithm POW(x, n):
           If n == 0: Return 1
           If n < 0:
               x = 1 / x
               n = -n

           temp = POW(x, n/2)
           If n % 2 == 0:
               Return temp * temp
           Else:
               Return temp * temp * x
```

Java Implementation

```
class Solution {
           public double myPow(double x, int n) {
               if (n == 0) return 1.0;
               if (n < 0) {
                   x = 1 / x;
                   // Handle Integer.MIN_VALUE overflow
                   if (n == Integer.MIN_VALUE) {
                        return x * myPow(x, -(n + 1));
                   }
                   n = -n;
               }

                 double temp = myPow(x, n / 2);
                 if (n % 2 == 0) {
                     return temp * temp;
                 } else {

                   return temp * temp * x;
               }
           }
       }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(log n)

128. Count Good Numbers

Understanding the Problem NEW

Count how many n-digit numbers exist where every digit at an even index (0, 2, 4...) is one of the 5 even digits (0,2,4,6,8), and every digit at an odd index is one of the 4 prime digits (2,3,5,7). Since this count can be astronomically large, the answer is typically required modulo 10^9+7.

Brute Force Approach NEW

A beginner might try to actually generate every valid n-digit number and count them — completely impossible for even moderately large n, since the count grows exponentially.

Why We Optimize NEW

You don't need to generate anything — this is purely a counting problem, and the count has a simple closed form: since even positions independently have 5 choices and odd positions independently have 4 choices, the total is `5^(number of even positions) × 4^(number of odd positions)`. Computing these two large powers efficiently reuses the fast exponentiation trick from problem 127 (with modular arithmetic applied at each squaring step to keep numbers manageable), turning what looks like a combinatorial explosion into two O(log n) power computations.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count numbers of length n where even positions have even digits and odd positions have prime digits.

Visual Diagram

```
n = 1 (single digit):
       Positions: 0 (even)
       Even positions: can be 0,2,4,6,8 → 5 options
       Total = 5

       n = 2:
       Position 0 (even): 5 options (0,2,4,6,8)
```

```
Position 1 (odd): 4 options (2,3,5,7)
Total = 5 * 4 = 20

n = 3:
Even positions: 5 options each
Odd positions: 4 options each
Total = 5^(ceil(n/2)) * 4^(floor(n/2))

For n=3:
5^2 * 4^1 = 25 * 4 = 100
```

Solution Approach

Concept: Power calculation with modular arithmetic.

Algorithm

```
Algorithm GOOD_NUMBERS(n):
          MOD = 10^9 + 7
          even_positions = (n + 1) / 2
          odd_positions = n / 2
          Return (pow(5, even_positions) * pow(4, odd_positions)) % MOD
```

Java Implementation

```
class Solution {
          private static final int MOD = 1_000_000_007;

          public int countGoodNumbers(long n) {
              long even = (n + 1) / 2;
              long odd = n / 2;
              return (int)(pow(5, even) * pow(4, odd) % MOD);
          }

          private long pow(long base, long exp) {
              long result = 1;
              while (exp > 0) {
                  if (exp % 2 == 1) {
                      result = (result * base) % MOD;
                  }
                  base = (base * base) % MOD;
                  exp /= 2;
              }
              return result;
          }
      }
```

Time & Space Complexity

Time Complexity: O(log n) | Space Complexity: O(1)

129. Sort a Stack Using Recursion

Understanding the Problem NEW

Sort a stack (a last-in-first-out structure) into order, using *only* recursion — no loops, and no other data structure like an array to dump the values into.

Brute Force Approach NEW

A beginner might reach for an array: pop every element off the stack into an array, sort the array (Chapter 1 style), and push the values back — this works, but it isn't really 'using recursion to sort' in the spirit intended, and it uses O(n) extra space for the array plus the cost of a general sort.

Why We Optimize NEW

The trick is to use the **call stack itself** as your only extra storage: recursively pop the top element off, recursively sort the rest of the (now smaller) stack first, and then insert the popped element back into its correct position within the now-sorted remainder — which itself requires a second small recursive helper that pops elements until it finds the right insertion point, inserts, and then pushes the popped elements back on top. This achieves the sort using only the stack itself and the recursion call stack (no separate array), at the cost of a fairly inefficient $O(n^2)$ time complexity, since each insertion may need to temporarily remove several elements.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Sort a stack using only recursion (no loops).

Visual Diagram

```
Stack: [5, 2, 8, 1, 3] (top)
       Sort: [1, 2, 3, 5, 8]

       Recursive steps:
       sort(stack):
         1. Pop top: 3
         2. Sort remaining: [5,2,8,1] → [1,2,5,8]
         3. Insert 3 in sorted order: [1,2,3,5,8]
```

```
      insert(stack, element):
        1. If stack empty or top <= element:
           Push element, return
        2. Pop top: 8
        3. Insert element in remaining
        4. Push 8 back
```

Solution Approach

Concept: Recursively pop and insert in sorted position.

Algorithm

```
Algorithm SORT_STACK(stack):
          If stack is empty: Return
          top = stack.pop()
          SORT_STACK(stack)
          INSERT_SORTED(stack, top)

      Algorithm INSERT_SORTED(stack, element):
          If stack empty or stack.peek() <= element:
              stack.push(element)
              Return
          top = stack.pop()
          INSERT_SORTED(stack, element)
          stack.push(top)
```

Java Implementation

```
class Solution {
          public void sortStack(Stack<Integer> stack) {
              if (stack.isEmpty()) return;

                int top = stack.pop();
                sortStack(stack);
                insertSorted(stack, top);
          }

          private void insertSorted(Stack<Integer> stack, int element) {
              if (stack.isEmpty() || stack.peek() <= element) {
                  stack.push(element);
                  return;
              }

                int top = stack.pop();
                insertSorted(stack, element);

              stack.push(top);
```

```
            }
        }
```

Time & Space Complexity

Time Complexity: O(n²) | Space Complexity: O(n)

130. Reverse a Stack Using Recursion

Understanding the Problem NEW

Reverse the order of elements in a stack using only recursion, again no explicit loops or auxiliary data structures like arrays.

Brute Force Approach NEW

As above, a beginner might use an auxiliary array or a second stack to reverse order and rebuild the stack — correct but doesn't showcase pure recursive manipulation.

Why We Optimize NEW

Recursively pop all elements off the stack (down to empty, via recursive calls), and then, as the recursion unwinds, insert each popped element at the very **bottom** of the stack instead of the top — which itself needs a small recursive helper (pop everything, recursively place the new element at the bottom once the stack is empty, then push everything back on top in the reverse order they were popped). The stack ends up reversed purely by controlling *where* each element gets reinserted during the unwind phase of recursion, with no extra array needed — just the recursion call stack itself as implicit storage.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Reverse a stack using only recursion.

Visual Diagram

```
Stack: [1, 2, 3, 4, 5] (top)
       Reverse: [5, 4, 3, 2, 1]

       Recursive steps:
       reverse(stack):
         1. Pop top: 1
         2. Reverse remaining: [5,4,3,2]
         3. Insert at bottom: [5,4,3,2,1]

       insertAtBottom(stack, element):
         1. If stack empty: Push element, return
```

```
2. Pop top: 2
3. Insert element at bottom of remaining
4. Push 2 back
```

Solution Approach

Concept: Pop all elements and insert at bottom.

Algorithm

```
Algorithm REVERSE_STACK(stack):
           If stack is empty: Return
           top = stack.pop()
           REVERSE_STACK(stack)
           INSERT_AT_BOTTOM(stack, top)

       Algorithm INSERT_AT_BOTTOM(stack, element):
           If stack is empty:
               stack.push(element)
               Return
           top = stack.pop()
           INSERT_AT_BOTTOM(stack, element)
           stack.push(top)
```

Java Implementation

```
class Solution {
           public void reverseStack(Stack<Integer> stack) {
               if (stack.isEmpty()) return;

                 int top = stack.pop();
                 reverseStack(stack);
                 insertAtBottom(stack, top);
           }

           private void insertAtBottom(Stack<Integer> stack, int element) {
               if (stack.isEmpty()) {
                   stack.push(element);
                   return;
               }

                 int top = stack.pop();
                 insertAtBottom(stack, element);
                 stack.push(top);

           }
       }
```

Time & Space Complexity

Time Complexity: $O(n^2)$ | Space Complexity: $O(n)$

131. Generate All Binary Strings

Understanding the Problem NEW

Generate every possible binary string (made of 0s and 1s) of a given length n. E.g. for n=2: `00, 01, 10, 11`.

Brute Force Approach NEW

You could compute this without recursion by counting from 0 to 2^n-1 in binary and formatting each number as an n-digit binary string — this actually works reasonably well, but doesn't build the recursive 'branching decision' pattern that's foundational for the harder backtracking problems ahead in this chapter.

Why We Optimize NEW

The recursive approach frames this as a series of decisions: at each position, you can either place a '0' *or* a '1', and recursively fill in the remaining positions for each choice — this creates a branching 'decision tree' with exactly 2 choices at each of the n levels, producing all 2^n combinations. This 'try each choice, recurse, then move on' pattern — sometimes just called exhaustive recursive generation — is the direct ancestor of the backtracking technique used extensively later in this chapter (subsequences, combination sum, N-Queens, and more), so it's worth internalizing the pattern here even though a non-recursive shortcut exists for this specific problem.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate all binary strings of length n.

Visual Diagram

```
n = 2
       [0,0]
       [0,1]
       [1,0]
       [1,1]

       Recursive tree:
                ""
```

```
              /    \
            "0"    "1"
           / \     / \
         "00" "01" "10" "11"

       n = 3 (8 strings):
       000, 001, 010, 011, 100, 101, 110, 111
```

Solution Approach

Concept: Recursively build strings.

Algorithm

```
Algorithm BINARY_STRINGS(n):
           result = []
           generate("", n, result)
           Return result

       Algorithm generate(current, n, result):
           If length(current) == n:
               result.add(current)
               Return
           generate(current + "0", n, result)
           generate(current + "1", n, result)
```

Java Implementation

```
class Solution {
           public List<String> generateBinaryStrings(int n) {
               List<String> result = new ArrayList<>();
               generate("", n, result);
               return result;
           }

            private void generate(String current, int n, List<String> result) {
                if (current.length() == n) {
                    result.add(current);
                    return;
                }

                 generate(current + "0", n, result);
                 generate(current + "1", n, result);
            }

       }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n * n)

132. Generate Parentheses

Understanding the Problem NEW

Generate every valid combination of n pairs of well-formed parentheses. E.g. for n=2: `(())`, `()()` — but not `)(()(` or other invalid arrangements.

Brute Force Approach NEW

A beginner might generate *all* possible strings of 2n parenthesis characters (open or close at each position, like problem 131's binary strings) and then filter out the invalid ones afterward by checking each generated string — this works but wastes a lot of effort generating and then discarding strings that could never have been valid in the first place.

Why We Optimize NEW

Instead of generating everything and filtering afterward, you can **prune invalid branches early** during the recursive decision-making itself: at each step, only add an opening bracket if you haven't used all n yet, and only add a closing bracket if doing so wouldn't create more closing brackets than opening ones seen so far. This is the essence of **backtracking** — track just enough state (count of opens used, count of closes used) to reject bad choices immediately, rather than generating a complete (possibly invalid) string and checking it only at the very end. This dramatically reduces wasted work compared to generate-then-filter.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate all valid combinations of n pairs of parentheses.

Visual Diagram

```
n = 1: ["()"]
       n = 2: ["(())", "()()"]
       n = 3: ["((()))", "(()())", "(())()", "()(())", "()()()"]

       Recursive tree (n=2):
                       ""
                       "("
                    /      \
```

```
                "(("      "()"
              /   \         |
            "((("  "(()"    "()("
              |      |        |
            "((()))" "(()())" "()()"
              |
            "((()))"
```

Solution Approach

Concept: Track open and close counts.

Algorithm

```
Algorithm GENERATE_PARENTHESES(n):
           result = []
           generate("", 0, 0, n, result)
           Return result

       Algorithm generate(current, open, close, n, result):
           If open == n and close == n:
               result.add(current)
               Return
           If open < n:
               generate(current + "(", open+1, close, n, result)
           If close < open:
               generate(current + ")", open, close+1, n, result)
```

Java Implementation

```
class Solution {
           public List<String> generateParenthesis(int n) {
               List<String> result = new ArrayList<>();
               generate("", 0, 0, n, result);
               return result;
           }

           private void generate(String current, int open, int close, int n,
List<String> result) {
               if (open == n && close == n) {
                   result.add(current);
                   return;
               }

                 if (open < n) {
                     generate(current + "(", open + 1, close, n, result);
                 }
                 if (close < open) {

                  generate(current + ")", open, close + 1, n, result);
```

```
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(4^n/√n) | Space Complexity: O(4^n/√n)

133. Print All Subsequences / Power Set

Understanding the Problem NEW

Print every possible subsequence (subset, keeping relative order) of an array — including the empty subsequence and the full array itself. For an array of size n, there are always exactly 2^n subsequences (the 'Power Set').

Brute Force Approach NEW

There isn't really a slower alternative to contrast against — generating all subsequences inherently requires visiting all 2^n possibilities, since that's the actual size of the answer. The main risk for beginners is an inefficient implementation (e.g. rebuilding a new list/array at every recursive call instead of reusing one shared list with add/remove).

Why We Optimize NEW

The clean recursive framing, closely related to the binary-strings idea in problem 131: for each element, make a binary decision — **include it, or exclude it** — and recurse on the rest of the array for both choices. Using a single shared temporary list that you append to before recursing and remove from after returning (rather than copying a new list at every level) keeps the extra work per call to O(1), so the total time is proportional to the total output size, $O(2^n \times n)$ (the n factor from copying each subsequence when you record it) — this 'include or exclude' framing recurs constantly for the rest of this chapter's subset/combination problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Print all subsequences of an array.

Visual Diagram

```
Array: [3, 1, 2]
       Subsequences:
       []
       [3]
       [1]
       [2]
```

```
       [3,1]
       [3,2]
       [1,2]
       [3,1,2]

       Recursive tree:
                    []
                /       \
              [3]           []
             /    \       /    \
           [3,1] [3]    [1]    []
          / \     / \ / \ / \
       [3,1,2][3,1][3,2][3][1,2][1][2][]
```

Solution Approach

Concept: Include or exclude each element.

Algorithm

```
Algorithm SUBSETS(arr):
           result = []
           generate(0, [], arr, result)
           Return result

       Algorithm generate(index, current, arr, result):
           If index == n:
               result.add(new ArrayList(current))
               Return
           // Exclude
           generate(index+1, current, arr, result)
           // Include
           current.add(arr[index])
           generate(index+1, current, arr, result)
           current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> subsets(int[] nums) {
               List<List<Integer>> result = new ArrayList<>();
               generate(0, new ArrayList<>(), nums, result);
               return result;
           }

           private void generate(int index, List<Integer> current, int[] nums,
List<List<Integer>> res
               if (index == nums.length) {
                   result.add(new ArrayList<>(current));
                   return;
```

```
            }

            // Exclude
            generate(index + 1, current, nums, result);

            // Include
            current.add(nums[index]);
            generate(index + 1, current, nums, result);
            current.remove(current.size() - 1);
        }
    }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n * n)

134. All Patterns of Subsequences (Print, Count, Check)

Understanding the Problem NEW

This is really three closely related variations built on the exact same include/exclude recursion from problem 133: (1) **print** every subsequence, (2) **count** how many subsequences satisfy some condition, and (3) **check** whether at least one subsequence satisfies some condition.

Brute Force Approach NEW

Treating these as three separate problems requiring three separate solutions would be wasteful — they share almost identical recursive structure.

Why We Optimize NEW

The valuable lesson here is recognizing that the *same* include/exclude recursion tree can be adapted for different purposes just by changing what happens at the base case and what gets returned: printing accumulates and outputs each valid path, counting adds 1 (or the sum of two recursive counts) instead of printing, and existence-checking can **short-circuit** — returning true immediately the moment any valid path is found, instead of continuing to explore the rest of the tree. That short-circuit for existence checks is a genuine optimization over the print/count versions, since you can stop early instead of exploring all 2^n branches.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Learn patterns: print, count, check existence.

Visual Diagram

```
Array: [1, 2, 1], K=2

       Pattern 1: Print all subsequences with sum K
       [2], [1,1]

       Pattern 2: Count subsequences with sum K
       [2], [1,1] → Count = 2
```

```
Pattern 3: Check if any subsequence with sum K exists
Yes (2 exists)

Recursive patterns:
// Print
if (index == n && sum == K) print

// Count
if (index == n) return sum == K ? 1 : 0
return count(index+1, sum) + count(index+1, sum+arr[index])

// Check
if (index == n) return sum == K
if (check(index+1, sum)) return true
if (check(index+1, sum+arr[index])) return true
return false
```

Solution Approach

Concept: Three variations of subsequence generation.

Java Implementation

```
class Solution {
           // Print all subsequences with sum K
           public void printSubsequences(int[] arr, int K) {
               print(0, 0, new ArrayList<>(), arr, K);
           }

           private void print(int index, int sum, List<Integer> current, int[] arr,
int K) {
               if (index == arr.length) {
                   if (sum == K) {
                       System.out.println(current);
                   }
                   return;
               }

                // Exclude
                print(index + 1, sum, current, arr, K);

                // Include
                current.add(arr[index]);
                print(index + 1, sum + arr[index], current, arr, K);
                current.remove(current.size() - 1);
           }

           // Count subsequences with sum K

          public int countSubsequences(int[] arr, int K) {
              return count(0, 0, arr, K);
```

```
        }

        private int count(int index, int sum, int[] arr, int K) {
            if (index == arr.length) {
                return sum == K ? 1 : 0;
            }
            return count(index + 1, sum, arr, K) + count(index + 1, sum +
arr[index], arr, K);
        }

        // Check if any subsequence with sum K exists
        public boolean existsSubsequence(int[] arr, int K) {
            return exists(0, 0, arr, K);
        }

        private boolean exists(int index, int sum, int[] arr, int K) {
            if (index == arr.length) {
                return sum == K;
            }
            if (exists(index + 1, sum, arr, K)) return true;
            if (exists(index + 1, sum + arr[index], arr, K)) return true;
            return false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

135. Count All Subsequences with Sum K

Understanding the Problem NEW

Count how many subsequences of the array have elements summing to exactly K.

Brute Force Approach NEW

This is direct: use the include/exclude recursion, and at each 'leaf' of the decision tree (once every element has been decided), check if the accumulated sum equals K — this correctly counts all valid subsequences, but explores the full 2^n tree regardless of whether a sum could still possibly reach K.

Why We Optimize NEW

A useful, cheap optimization here is to prune early: if the running sum has already exceeded K partway through building a subsequence (assuming non-negative numbers), there's no point continuing to include more elements in that branch — you can stop exploring it immediately rather than waiting until the base case. This kind of early pruning doesn't change the theoretical worst-case complexity (still $O(2^n)$ in the worst case) but can meaningfully cut down real-world running time by avoiding provably-doomed branches — a preview of the pruning ideas used more aggressively in problems 137-141.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Count number of subsequences with sum equal to K.

Visual Diagram

```
Array: [1, 2, 1], K=2
      Subsequences with sum 2:
      [2] (index 1)
      [1,1] (index 0,2)
      Count = 2

      Recursive approach:
      count(0, 0):
        count(1, 0):
          count(2, 0):
```

```
            count(3, 0): sum=0 != 2 → 0
            count(3, 1): sum=1 != 2 → 0
          count(2, 1):
            count(3, 1): sum=1 != 2 → 0
            count(3, 2): sum=2 == 2 → 1
        count(1, 1):
          count(2, 1):
            count(3, 1): sum=1 != 2 → 0
            count(3, 2): sum=2 == 2 → 1
          count(2, 2):
            count(3, 2): sum=2 == 2 → 1
            count(3, 3): sum=3 != 2 → 0
      Total = 2
```

Solution Approach

Concept: Standard subsequence count with sum.

Algorithm

```
Algorithm COUNT_SUM_K(arr, K):
          Return count(0, 0, arr, K)

      Algorithm count(index, sum, arr, K):
          If index == n:
              Return sum == K ? 1 : 0
          // Exclude
          exclude = count(index+1, sum, arr, K)
          // Include
          include = count(index+1, sum+arr[index], arr, K)
          Return exclude + include
```

Java Implementation

```
class Solution {
          public int countSubsequencesWithSumK(int[] arr, int K) {
              return count(0, 0, arr, K);
          }

          private int count(int index, int sum, int[] arr, int K) {
              if (index == arr.length) {
                  return sum == K ? 1 : 0;
              }

                 // Exclude current element

              int exclude = count(index + 1, sum, arr, K);
              // Include current element
              int include = count(index + 1, sum + arr[index], arr, K);
```

```
            return exclude + include;
        }
    }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

136. Check if There Exists a Subsequence with Sum K

Understanding the Problem NEW

Check whether *any* subsequence of the array sums to exactly K (a yes/no answer, not a count).

Brute Force Approach NEW

Using the same include/exclude recursion and checking every leaf for a matching sum works, but explores the entire 2^n tree even after finding an answer, if you don't explicitly stop.

Why We Optimize NEW

Since you only need to know if *any* valid subsequence exists, the recursive calls should **return true immediately** the moment one branch reports success, rather than dutifully continuing to explore every remaining branch. This 'short-circuit on success' idea (mentioned in problem 134) can turn a full $O(2^n)$ exploration into something that finishes much earlier in practice, especially if a valid subsequence appears early in the search order — even though the worst case (no valid subsequence exists at all) still requires exploring everything to be sure.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Check if any subsequence sums to K.

Visual Diagram

```
Array: [1, 2, 1], K=3
       Subsequence [1,2] sum=3 exists → true

       Early termination optimization:
       exists(0, 0):
         exists(1, 0):
           exists(2, 0):
             exists(3, 0): false
             exists(3, 1): false
```

```
          exists(2, 1):
            exists(3, 1): false
            exists(3, 2): false
        exists(1, 1):
          exists(2, 1):
            exists(3, 1): false
            exists(3, 2): false
          exists(2, 2):
            exists(3, 2): false
            exists(3, 3): true → return true
```

Solution Approach

Concept: Return early when found.

Algorithm

```
Algorithm EXISTS_SUM_K(arr, K):
          Return exists(0, 0, arr, K)

      Algorithm exists(index, sum, arr, K):
          If index == n:
              Return sum == K
          If exists(index+1, sum, arr, K):
              Return true
          If exists(index+1, sum+arr[index], arr, K):
              Return true
          Return false
```

Java Implementation

```
class Solution {
          public boolean existsSubsequenceWithSumK(int[] arr, int K) {
              return exists(0, 0, arr, K);
          }

          private boolean exists(int index, int sum, int[] arr, int K) {
              if (index == arr.length) {
                  return sum == K;
              }

                // Exclude
                if (exists(index + 1, sum, arr, K)) return true;
                // Include
                if (exists(index + 1, sum + arr[index], arr, K)) return true;
                return false;

          }
     }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

137. Combination Sum

Understanding the Problem NEW

Find all unique combinations of numbers from a given array that sum to a target, where the **same number can be reused multiple times**. E.g. array `[2,3,6,7]`, target 7 → `[7]` and `[2,2,3]` are both valid.

Brute Force Approach NEW

Trying to generate every possible combination without any structure risks both missing valid combinations and generating massive numbers of duplicate combinations (like `[2,3,2]` and `[2,2,3]` being counted as different when they're really the same set).

Why We Optimize NEW

The key structural decision that avoids duplicates: at each recursive step, when you choose to include an element again, you allow the *same index* to be revisited (permitting reuse), but when you choose to move to a *different* element, you only ever move forward in the array (never backward) — this ordering rule guarantees every valid combination is generated in exactly one canonical order, never as a rearranged duplicate. Combined with early pruning (stop exploring a branch once the running sum exceeds the target, assuming positive numbers), this keeps the search efficient despite allowing repeated elements.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all combinations that sum to target (unlimited use of elements).

Visual Diagram

```
Array: [2, 3, 5], Target: 8

       Combinations:
       [2,2,2,2] → 8 ✓
       [2,3,3] → 8 ✓
       [3,5] → 8 ✓

       Recursive tree:
```

```
                    []
              /      |      \
            [2]     [3]    [5]
           / \     / \     / \
         [2,2] [2,3] [3,3] [3,5] [5,5]
        / \    / \     / \    / \
      [2,2,2] [2,2,3] [2,3,3] [3,3,3] ...
```

Solution Approach

Concept: Allow repeated elements, maintain index.

Algorithm

```
Algorithm COMBINATION_SUM(candidates, target):
           result = []
           Arrays.sort(candidates)
           findCombinations(0, target, [], result, candidates)
           Return result

       Algorithm findCombinations(index, remaining, current, result, candidates):
           If remaining == 0:
               result.add(new ArrayList(current))
               Return
           For i = index to n-1:
               If candidates[i] > remaining: Break
               current.add(candidates[i])
               findCombinations(i, remaining-candidates[i], current, result,
candidates)
               current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> combinationSum(int[] candidates, int target)
{
               List<List<Integer>> result = new ArrayList<>();
               Arrays.sort(candidates);
               findCombinations(0, target, new ArrayList<>(), result, candidates);
               return result;
           }

           private void findCombinations(int index, int remaining, List<Integer>
current,
                                         List<List<Integer>> result, int[]
candidates) {
               if (remaining == 0) {
                   result.add(new ArrayList<>(current));
                   return;
               }
```

```
                for (int i = index; i < candidates.length; i++) {

                    if (candidates[i] > remaining) break;

                    current.add(candidates[i]);
                    findCombinations(i, remaining - candidates[i], current, result,
candidates);
                    current.remove(current.size() - 1);
                }
            }
        }
```

Time & Space Complexity

Time Complexity: O(2^(target/min)) | Space Complexity: O(target/min)

138. Combination Sum-II

Understanding the Problem NEW

Same as Combination Sum, but now each number in the array can be used **at most once**, and the array may contain duplicate values that still shouldn't produce duplicate combinations in the output.

Brute Force Approach NEW

Generating all combinations without any duplicate-avoidance strategy and then filtering out duplicate combinations afterward (e.g. via a hashset of sorted tuples) works, but wastes significant effort generating and then discarding a large number of duplicate branches.

Why We Optimize NEW

The standard fix: **sort the array first**, then during the recursive exploration, whenever you're deciding whether to include the current element at a given recursion depth, skip over it if it's identical to the element you just skipped at the *same* depth (not the same value used earlier at a different depth — that's fine). This 'skip duplicates at the same recursion level, after sorting' pattern is exactly the same duplicate-avoidance trick used for 3-Sum and 4-Sum in the Arrays chapter, just applied to a recursive search instead of a two-pointer sweep — and it prevents duplicate combinations from ever being generated in the first place, rather than filtering them out after the fact.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find unique combinations (no repeated elements, each used once).

Visual Diagram

```
Array: [10,1,2,7,6,1,5], Target: 8
       Unique combinations:
       [1,1,6]
       [1,2,5]
       [1,7]
       [2,6]
```

```
       Recursive approach:
       1. Sort: [1,1,2,5,6,7,10]
       2. Skip duplicates at same level
       3. Each element can be used once

       Combination [1,1,6]:
       Index 0: use 1
       Index 1: use 1 (skip duplicate)
       Index 5: use 6
```

Solution Approach

Concept: Sort and skip duplicates.

Algorithm

```
Algorithm COMBINATION_SUM_II(candidates, target):
           Arrays.sort(candidates)
           result = []
           findCombinations(0, target, [], result, candidates)
           Return result

       Algorithm findCombinations(index, remaining, current, result, candidates):
           If remaining == 0:
               result.add(new ArrayList(current))
               Return
           For i = index to n-1:
               If i > index and candidates[i] == candidates[i-1]:
                   Continue
               If candidates[i] > remaining: Break
               current.add(candidates[i])
               findCombinations(i+1, remaining-candidates[i], current, result,
candidates)
               current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> combinationSum2(int[] candidates, int target)
{
               List<List<Integer>> result = new ArrayList<>();
               Arrays.sort(candidates);
               findCombinations(0, target, new ArrayList<>(), result, candidates);
               return result;
           }

           private void findCombinations(int index, int remaining, List<Integer>
current,
                                         List<List<Integer>> result, int[]
candidates) {
```

```
                if (remaining == 0) {
                    result.add(new ArrayList<>(current));

                    return;
                }

                for (int i = index; i < candidates.length; i++) {
                    // Skip duplicates
                    if (i > index && candidates[i] == candidates[i - 1]) continue;
                    if (candidates[i] > remaining) break;

                    current.add(candidates[i]);
                    findCombinations(i + 1, remaining - candidates[i], current,
result, candidates);
                    current.remove(current.size() - 1);
                }
            }
        }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

139. Subset Sum-I

Understanding the Problem NEW

Find all possible sums achievable by any subset of the array (not the subsets themselves, just their sums). This is a direct application of the include/exclude recursion from problem 133, now tracking sums instead of the elements.

Brute Force Approach NEW

There isn't a slower alternative to contrast — this problem inherently requires exploring all 2^n subsets to enumerate every possible sum, since any one of them could produce a sum not achievable another way.

Why We Optimize NEW

Straightforward include/exclude recursion, adding the current element's value to a running sum when included, and recording the final sum at each leaf of the tree — $O(2^n)$ time since there are that many subsets to enumerate. The main practical note: if duplicate sums are undesirable in the output, sorting the resulting list of sums at the end (or using a set) removes them, though the array itself has no ordering requirement here (unlike problem 140).

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all possible subset sums.

Visual Diagram

```
Array: [3, 1, 2]
       Subset sums:
       [] → 0
       [3] → 3
       [1] → 1
       [2] → 2
       [3,1] → 4
       [3,2] → 5
       [1,2] → 3
       [3,1,2] → 6

       Sorted sums: [0, 1, 2, 3, 3, 4, 5, 6]
```

```
       Recursive:
       subsetSum(0, 0):
         exclude: subsetSum(1, 0)
           exclude: subsetSum(2, 0)
             exclude: subsetSum(3, 0) → add 0
             include: subsetSum(3, 3) → add 3
           include: subsetSum(2, 1)
             exclude: subsetSum(3, 1) → add 1
             include: subsetSum(3, 4) → add 4
         include: subsetSum(1, 3)
           exclude: subsetSum(2, 3)
             exclude: subsetSum(3, 3) → add 3
             include: subsetSum(3, 6) → add 6
           include: subsetSum(2, 4)
             exclude: subsetSum(3, 4) → add 4
             include: subsetSum(3, 7) → add 7
```

Solution Approach

Concept: Include/exclude elements.

Algorithm

```
Algorithm SUBSET_SUMS(arr):
           result = []
           generate(0, 0, arr, result)
           Collections.sort(result)
           Return result

       Algorithm generate(index, sum, arr, result):
           If index == n:
               result.add(sum)
               Return
           // Exclude
           generate(index+1, sum, arr, result)
           // Include
           generate(index+1, sum+arr[index], arr, result)
```

Java Implementation

```
class Solution {
           public List<Integer> subsetSums(int[] arr) {

              List<Integer> result = new ArrayList<>();
              generate(0, 0, arr, result);
              Collections.sort(result);
              return result;
          }
```

```
          private void generate(int index, int sum, int[] arr, List<Integer>
result) {
              if (index == arr.length) {
                  result.add(sum);
                  return;
              }

              // Exclude
              generate(index + 1, sum, arr, result);
              // Include
              generate(index + 1, sum + arr[index], arr, result);
          }
      }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n)

140. Subset Sum-II

Understanding the Problem NEW

Same idea as Subset Sum-I, but now find the unique *subsets themselves* (not just sums), where the array may contain duplicate values that shouldn't produce duplicate subsets in the output.

Brute Force Approach NEW

Generate every subset via plain include/exclude recursion and then deduplicate the results afterward using a hashset of sorted subsets — correct, but wastes effort generating duplicates before throwing them away.

Why We Optimize NEW

Exactly the same fix as Combination Sum-II (problem 138): sort the array first, then at each recursion level, skip over duplicate values at that *same* level (not across different levels) to avoid ever generating the same subset twice. This is the third appearance of the 'sort, then skip same-level duplicates' pattern in this chapter — worth recognizing as one reusable technique rather than three separate ones.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find unique subsets (Power Set without duplicates).

Visual Diagram

```
Array: [1, 2, 2]
       Unique subsets:
       []
       [1]
       [2]
       [1,2]
       [2,2]
       [1,2,2]

       Duplicate handling:
       At each level, skip duplicate elements
       Index 0: use 1, skip 2 duplicate
```

```
       Index 1: use first 2
       Index 2: use second 2
```

Solution Approach

Concept: Sort and skip duplicates at each level.

Algorithm

```
Algorithm UNIQUE_SUBSETS(arr):
           Arrays.sort(arr)
           result = []
           generate(0, [], arr, result)
           Return result

       Algorithm generate(index, current, arr, result):
           result.add(new ArrayList(current))
           For i = index to n-1:
               If i > index and arr[i] == arr[i-1]:
                   Continue
               current.add(arr[i])
               generate(i+1, current, arr, result)
               current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> subsetsWithDup(int[] nums) {
               List<List<Integer>> result = new ArrayList<>();
               Arrays.sort(nums);
               generate(0, new ArrayList<>(), nums, result);
               return result;
           }

           private void generate(int index, List<Integer> current, int[] nums,
List<List<Integer>> res
               result.add(new ArrayList<>(current));

                 for (int i = index; i < nums.length; i++) {
                     if (i > index && nums[i] == nums[i - 1]) continue;

                    current.add(nums[i]);
                    generate(i + 1, current, nums, result);
                    current.remove(current.size() - 1);

               }
           }
       }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(2^n * n)

141. Combination Sum - III

Understanding the Problem NEW

Find all combinations of exactly K numbers (each from 1-9, each used at most once) that sum to a given target n. E.g. k=3, n=7 → `[1,2,4]`.

Brute Force Approach NEW

Since the numbers are fixed to 1-9, you could enumerate every possible k-sized combination from those 9 digits and check which ones sum to n — with only 9 possible digits this is small enough to brute-force reasonably, but it's still cleaner to build the combination incrementally with pruning.

Why We Optimize NEW

This is a more constrained version of Combination Sum (problem 137): because digits can't repeat and only 9 exist, and you need exactly k of them, you can prune very aggressively during the recursive search — stop exploring a branch the moment the running sum exceeds n, the moment you've already picked k numbers (whether or not the sum matches, in which case you check and record or discard), or the moment there aren't enough remaining digits left to reach k total picks. These early exits keep the search fast despite the exhaustive-looking structure, since the small fixed digit range (1-9) means the search tree is naturally shallow and narrow.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find all combinations of k numbers that sum to n (1-9 each used once).

Visual Diagram

```
k = 3, n = 7
       Combinations:
       [1,2,4] → 7 ✓

       k = 3, n = 9
       [1,2,6]
       [1,3,5]
       [2,3,4]
```

```
       Recursive:
       combination(1, 9, 3):
         i=1: combination(2, 8, 2)
           i=2: combination(3, 6, 1)
             i=3: combination(4, 3, 0) → not valid
             i=4: combination(5, 2, 0) → not valid
             i=5: combination(6, 1, 0) → not valid
             i=6: combination(7, 0, 0) → [1,2,6] ✓
           i=3: combination(4, 5, 1)
             i=4: combination(5, 1, 0) → not valid
             i=5: combination(6, 0, 0) → [1,3,5] ✓
         i=2: combination(3, 7, 2)
           i=3: combination(4, 4, 1)
             i=4: combination(5, 0, 0) → [2,3,4] ✓
```

Solution Approach

Concept: Generate combinations with fixed length.

Algorithm

```
Algorithm COMBINATION_SUM_3(k, n):
           result = []
           findCombinations(1, k, n, [], result)
           Return result

       Algorithm findCombinations(start, remaining_k, remaining_sum, current,
result):
           If remaining_k == 0 and remaining_sum == 0:
               result.add(new ArrayList(current))
               Return
           If remaining_k == 0 or remaining_sum < 0:
               Return
           For i = start to 9:
               If i > remaining_sum: Break
               current.add(i)
               findCombinations(i+1, remaining_k-1, remaining_sum-i, current,
result)
               current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<List<Integer>> combinationSum3(int k, int n) {
               List<List<Integer>> result = new ArrayList<>();
               findCombinations(1, k, n, new ArrayList<>(), result);
               return result;
           }
```

```
            private void findCombinations(int start, int remainingK, int
remainingSum,
                                          List<Integer> current,
List<List<Integer>> result) {
                if (remainingK == 0 && remainingSum == 0) {
                    result.add(new ArrayList<>(current));
                    return;
                }
                if (remainingK == 0 || remainingSum < 0) {
                    return;
                }

                for (int i = start; i <= 9; i++) {
                    if (i > remainingSum) break;

                    current.add(i);
                    findCombinations(i + 1, remainingK - 1, remainingSum - i,
current, result);
                    current.remove(current.size() - 1);
                }
            }
        }
```

Time & Space Complexity

Time Complexity: O(9!/(9-k)!) | Space Complexity: O(k)

142. Letter Combinations of a Phone Number

Understanding the Problem NEW

Given a string of digits (like an old phone keypad, e.g. '23'), where each digit maps to a set of letters (2→abc, 3→def), generate every possible letter combination the digit string could represent. E.g. '23' → `ad, ae, af, bd, be, bf, cd, ce, cf`.

Brute Force Approach NEW

You could compute the total number of combinations in advance and try to generate them with nested loops, one loop per digit — this works but becomes unwieldy since the number of digits (and therefore nested loops) isn't fixed in advance.

Why We Optimize NEW

Recursion naturally handles a variable, unknown-in-advance number of 'positions to fill': at each recursive call, you're deciding which letter to place at the *current* digit's position, trying every letter that digit maps to, and recursing to handle the remaining digits — building up one combination per full root-to-leaf path through the recursion tree. This avoids needing to write a different number of nested loops depending on the input string's length, letting recursion adapt naturally to any length input.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Generate all letter combinations from phone digits.

Visual Diagram

```
Digits: "23"
       2 → a,b,c
       3 → d,e,f

       Combinations:
       ad, ae, af, bd, be, bf, cd, ce, cf

       Recursive tree:
```

```
             ""
          /  |   \
         a   b    c
        /|\ /|\ /|\
       ad ae af bd be bf cd ce cf
```

Solution Approach

Concept: Map digits to letters, backtrack.

Algorithm

```
Algorithm LETTER_COMBINATIONS(digits):
           If digits empty: Return []
           map = {"2":"abc", "3":"def", ...}
           result = []
           generate(0, "", digits, map, result)
           Return result

       Algorithm generate(index, current, digits, map, result):
           If index == len(digits):
               result.add(current)
               Return
           For each char in map[digits[index]]:
               generate(index+1, current+char, digits, map, result)
```

Java Implementation

```
class Solution {
           public List<String> letterCombinations(String digits) {
               if (digits.isEmpty()) return new ArrayList<>();

                 String[] mapping = {"", "", "abc", "def", "ghi", "jkl", "mno",
"pqrs", "tuv", "wxyz"};
                 List<String> result = new ArrayList<>();
                 generate(0, "", digits, mapping, result);
                 return result;
            }

            private void generate(int index, String current, String digits,
                                  String[] mapping, List<String> result) {
                if (index == digits.length()) {
                    result.add(current);
                    return;
                }

                 int digit = digits.charAt(index) - '0';
                 for (char c : mapping[digit].toCharArray()) {

                  generate(index + 1, current + c, digits, mapping, result);
```

```
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(4^n) | Space Complexity: O(4^n)

143. Palindrome Partitioning

Understanding the Problem NEW

Partition a string into pieces such that every piece is itself a palindrome, and find every way to do this. E.g. `"aab"` → `["a","a","b"]` and `["aa","b"]` are both valid partitions.

Brute Force Approach NEW

A beginner might try to generate every possible way to split the string into pieces first (all possible partition boundaries) and then filter out the ones where some piece isn't a palindrome — this generates many invalid partitions before rejecting them.

Why We Optimize NEW

Instead, use backtracking with early pruning: at each recursive step, try every possible *prefix* of the remaining string, but only recurse further if that prefix is itself already a valid palindrome (checked immediately, before committing to it) — invalid prefixes are rejected right away rather than being included in a full partition that gets checked only at the very end. This is the same 'prune invalid choices as early as possible' philosophy from Generate Parentheses (problem 132), applied here to partition boundaries instead of bracket counts.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Partition string into palindromic substrings.

Visual Diagram

```
Input: "aab"
       Partitions:
       ["a", "a", "b"]
       ["aa", "b"]

       Recursive tree:
                   "aab"
                 /       \
               "a"       "aa"
              /   \        |
```

```
          "a"    "ab"     "b"
          /        |
        "b"      "b"
```

Solution Approach

Concept: Check palindrome and partition.

Algorithm

```
Algorithm PALINDROME_PARTITION(s):
           result = []
           partition(0, [], s, result)
           Return result

       Algorithm partition(start, current, s, result):
           If start == n:
               result.add(new ArrayList(current))
               Return
           For end = start to n-1:
               If isPalindrome(s, start, end):
                   current.add(s.substring(start, end+1))
                   partition(end+1, current, s, result)
                   current.remove(current.size()-1)

       Algorithm isPalindrome(s, left, right):
           While left < right:
               If s[left] != s[right]: Return false
               left++; right--
           Return true
```

Java Implementation

```
class Solution {
           public List<List<String>> partition(String s) {
               List<List<String>> result = new ArrayList<>();
               partition(0, new ArrayList<>(), s, result);
               return result;
           }

            private void partition(int start, List<String> current, String s,
                                   List<List<String>> result) {

               if (start == s.length()) {
                   result.add(new ArrayList<>(current));
                   return;
               }

               for (int end = start; end < s.length(); end++) {
                   if (isPalindrome(s, start, end)) {
```

```
                    current.add(s.substring(start, end + 1));
                    partition(end + 1, current, s, result);
                    current.remove(current.size() - 1);
                }
            }
        }

        private boolean isPalindrome(String s, int left, int right) {
            while (left < right) {
                if (s.charAt(left) != s.charAt(right)) return false;
                left++;
                right--;
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n * 2^n) | Space Complexity: O(n)

144. Word Search

Understanding the Problem NEW

Given a 2D grid of letters and a target word, determine if the word can be traced out by moving between horizontally/vertically adjacent cells, using each cell at most once in a single trace.

Brute Force Approach NEW

Trying every possible starting cell and blindly exploring every direction without remembering which cells are already used in the current path could revisit the same cell twice within one trace, producing incorrect matches.

Why We Optimize NEW

This needs **DFS with backtracking and a 'visited' marker**: from each candidate starting cell matching the word's first letter, recursively try all four directions for the next letter, temporarily marking the current cell as visited (so it can't be reused later in this same path) — and crucially, **un-marking** it again once that recursive branch is done exploring (this is the 'backtrack' step), so the cell remains available for other paths that don't go through it. Forgetting to un-mark cells after backtracking out of a branch is the single most common bug in this kind of grid search — the search must restore the grid's original state before trying the next possibility.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find if word exists in a grid by adjacent cells.

Visual Diagram

```
Grid:
       [['A','B','C','E'],
        ['S','F','C','S'],
        ['A','D','E','E']]

       Word: "ABCCED"
       Path: (0,0)→(0,1)→(0,2)→(1,2)→(2,2)→(2,3)
       A→B→C→C→E→D
```

```
Search:
Start at (0,0) 'A'
  → (0,1) 'B'
    → (0,2) 'C'
      → (1,2) 'C'
        → (2,2) 'E'
          → (2,3) 'D' ✓
```

Solution Approach

Concept: DFS with backtracking and visited marking.

Algorithm

```
Algorithm WORD_SEARCH(board, word):
           For each cell:
               If board[i][j] == word[0] and dfs(i, j, 0):
                   Return true
           Return false

       Algorithm dfs(i, j, index):
           If index == len(word): Return true
           If out of bounds: Return false
           If board[i][j] != word[index]: Return false

            char = board[i][j]
            board[i][j] = '#' // Mark visited
            // Check all 4 directions
            if dfs(i+1,j,index+1) or dfs(i-1,j,index+1) or
               dfs(i,j+1,index+1) or dfs(i,j-1,index+1):
                Return true
            board[i][j] = char // Backtrack
            Return false
```

Java Implementation

```
class Solution {
           public boolean exist(char[][] board, String word) {
               int m = board.length, n = board[0].length;

                 for (int i = 0; i < m; i++) {
                     for (int j = 0; j < n; j++) {
                         if (board[i][j] == word.charAt(0) && dfs(i, j, 0, board,
word)) {
                             return true;
                         }
                     }

             }
```

```
                return false;
            }

            private boolean dfs(int i, int j, int index, char[][] board, String word)
{
                if (index == word.length()) return true;
                if (i < 0 || i >= board.length || j < 0 || j >= board[0].length)
return false;
                if (board[i][j] != word.charAt(index)) return false;

                char temp = board[i][j];
                board[i][j] = '#';

                boolean found = dfs(i+1, j, index+1, board, word) ||
                                dfs(i-1, j, index+1, board, word) ||
                                dfs(i, j+1, index+1, board, word) ||
                                dfs(i, j-1, index+1, board, word);

                board[i][j] = temp;
                return found;
            }
        }
```

Time & Space Complexity

Time Complexity: O(m*n*4^L) | Space Complexity: O(L)

145. N Queen

Understanding the Problem NEW

Place N queens on an N×N chessboard such that no two queens attack each other (no shared row, column, or diagonal). Find all such arrangements.

Brute Force Approach NEW

Trying every possible placement of N queens among all N^2 squares (checking all $C(N^2, N)$ combinations for validity) is astronomically expensive even for modest N.

Why We Optimize NEW

The key structural insight: since no two queens can share a row, you only ever need to place **exactly one queen per row** — this alone eliminates a huge number of invalid possibilities by construction, rather than generating them and checking afterward. Then, placing queens row by row, at each row you only try columns that don't conflict with any queen already placed in an earlier row (checking column and both diagonals) — if a row has no valid column, you **backtrack** to the previous row and try a different column there instead. This 'place one at a time, backtrack on failure' approach explores only partially-valid board states, pruning the search space dramatically compared to generating full boards and validating them after the fact.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Place N queens on NxN board so no two attack each other.

Visual Diagram

```
N = 4 Queens:
       Q . . .
       . . Q .
       . . . Q
       . Q . .

       Board:
       0: Q . . .
       1: . . Q .
       2: . . . Q
```

```
3: . Q . .

Backtracking:
Row 0: Place at col 0
Row 1: Try col 0 (invalid), 1 (invalid), 2 (valid)
Row 2: Try col 0 (invalid), 1 (invalid), 2 (invalid), 3 (valid)
Row 3: Try col 0 (invalid), 1 (valid) ✓
```

Solution Approach

Concept: Place queens row by row with conflict checking.

Algorithm

```
Algorithm N_QUEEN(n):
           board = new char[n][n]
           result = []
           solve(0, board, result)
           Return result

       Algorithm solve(row, board, result):
           If row == n:
               result.add(createBoard(board))
               Return
           For col = 0 to n-1:
               If isSafe(row, col, board):
                   board[row][col] = 'Q'
                   solve(row+1, board, result)
                   board[row][col] = '.'

       Algorithm isSafe(row, col, board):
           Check column
           Check upper-left diagonal
           Check upper-right diagonal
```

Java Implementation

```
class Solution {
           public List<List<String>> solveNQueens(int n) {
               char[][] board = new char[n][n];
               for (int i = 0; i < n; i++) {
                   Arrays.fill(board[i], '.');
               }
               List<List<String>> result = new ArrayList<>();
               solve(0, board, result);

               return result;
           }

           private void solve(int row, char[][] board, List<List<String>> result) {
```

```
            if (row == board.length) {
                result.add(construct(board));
                return;
            }

            for (int col = 0; col < board.length; col++) {
                if (isSafe(row, col, board)) {
                    board[row][col] = 'Q';
                    solve(row + 1, board, result);
                    board[row][col] = '.';
                }
            }
        }

        private boolean isSafe(int row, int col, char[][] board) {
            // Check column
            for (int i = 0; i < row; i++) {
                if (board[i][col] == 'Q') return false;
            }

            // Check upper-left diagonal
            for (int i = row - 1, j = col - 1; i >= 0 && j >= 0; i--, j--) {
                if (board[i][j] == 'Q') return false;
            }

            // Check upper-right diagonal
            for (int i = row - 1, j = col + 1; i >= 0 && j < board.length; i--,
j++) {
                if (board[i][j] == 'Q') return false;
            }

            return true;
        }

        private List<String> construct(char[][] board) {
            List<String> result = new ArrayList<>();
            for (char[] row : board) {
                result.add(new String(row));
            }
            return result;
        }
    }
```

Time & Space Complexity

Time Complexity: O(n!) | Space Complexity: O(n²)

146. Rat in a Maze

Understanding the Problem NEW

Find a path for a rat to travel from the top-left corner to the bottom-right corner of a maze (grid), moving only through open cells, and avoiding cells already visited in the current path.

Brute Force Approach NEW

Similar to Word Search, blindly trying all four directions from every cell without tracking visited cells in the current path risks infinite loops (bouncing back and forth between two cells) or invalid paths that revisit the same cell.

Why We Optimize NEW

Same DFS-with-backtracking-and-visited-marking pattern as Word Search (problem 144): try each direction from the current cell, mark the cell as visited before recursing, and un-mark it when backtracking out of a dead end so other paths can still use that cell. This structural template — mark, recurse, unmark — is the standard shape for essentially all grid/maze pathfinding problems, and it's worth explicitly recognizing that problems 144 and 146 are really the same technique applied to two differently-flavoured problems.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Find path for rat from (0,0) to (n-1,n-1).

Visual Diagram

```
Maze:
        1 0 0 0
        1 1 0 1
        1 1 0 0
        0 1 1 1

        Path: D D R R D D
        (0,0)→(1,0)→(2,0)→(2,1)→(2,2)→(3,2)→(3,3)

        Backtracking:
        Start: (0,0)
```

```
          → Down: (1,0)
            → Down: (2,0)
              → Right: (2,1)
                → Right: (2,2)
                  → Down: (3,2)
                     → Down: (3,3) ✓
```

Solution Approach

Concept: DFS with movement directions.

Algorithm

```
Algorithm RAT_IN_MAZE(maze):
            n = maze.length
            path = []
            If solve(0, 0, maze, path):
                Return path
            Return []

        Algorithm solve(i, j, maze, path):
            If i == n-1 and j == n-1:
                return true
            If not safe: return false

            // Try all directions: D, R, U, L
            For each direction:
                path.add(dir)
                If solve(new_i, new_j, maze, path):
                    return true
                path.remove(path.size()-1)
            return false
```

Java Implementation

```
class Solution {
            public List<String> findPath(int[][] maze) {
                int n = maze.length;
                List<String> result = new ArrayList<>();
                if (maze[0][0] == 0) return result;

                  boolean[][] visited = new boolean[n][n];
                  solve(0, 0, maze, visited, "", result);
                  return result;

           }

           private void solve(int i, int j, int[][] maze, boolean[][] visited,
                              String path, List<String> result) {
               int n = maze.length;
```

```
            if (i == n - 1 && j == n - 1) {
                result.add(path);
                return;
            }

            visited[i][j] = true;

            // Down
            if (i + 1 < n && !visited[i+1][j] && maze[i+1][j] == 1) {
                solve(i + 1, j, maze, visited, path + "D", result);
            }
            // Right
            if (j + 1 < n && !visited[i][j+1] && maze[i][j+1] == 1) {
                solve(i, j + 1, maze, visited, path + "R", result);
            }
            // Up
            if (i - 1 >= 0 && !visited[i-1][j] && maze[i-1][j] == 1) {
                solve(i - 1, j, maze, visited, path + "U", result);
            }
            // Left
            if (j - 1 >= 0 && !visited[i][j-1] && maze[i][j-1] == 1) {
                solve(i, j - 1, maze, visited, path + "L", result);
            }

            visited[i][j] = false;
        }
    }
```

Time & Space Complexity

Time Complexity: O(4^(n²)) | Space Complexity: O(n²)

147. Word Break

Understanding the Problem NEW

Given a string and a dictionary of words, determine if the string can be segmented into a sequence of one or more dictionary words. E.g. `"leetcode"` with dictionary `["leet","code"]` → yes.

Brute Force Approach NEW

The direct recursive approach: try every possible prefix of the remaining string, and if it's a dictionary word, recursively check if the rest can also be segmented — this is correct, but without any memory of previous results, the same 'remaining substring' can be re-checked repeatedly across different branches of the recursion tree, leading to exponential blowup ($O(2^n)$ in the worst case) from redundant recomputation.

Why We Optimize NEW

The critical optimization: whenever you compute whether a particular *starting index* in the string can be successfully segmented, **cache that result** (in an array or hashmap keyed by index) so that if the same starting index comes up again in a different branch of the recursion, you return the cached answer instantly instead of recomputing it from scratch. This is a form of **memoization** — turning what would otherwise be exponential repeated work into roughly $O(n^2)$ (one computation per starting index, each costing up to $O(n)$ to try every prefix) by remembering answers to subproblems you've already solved, a preview of dynamic programming ideas explored more fully in the DP chapter ahead.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Break string into words from dictionary.

Visual Diagram

```
s = "leetcode", wordDict = ["leet", "code"]
       Result: ["leet code"]

       s = "catsanddog", wordDict = ["cat","cats","and","sand","dog"]
       Results: ["cat sand dog", "cats and dog"]
```

```
      Recursive:
      "leetcode":
        "leet" + "code" → ["leet","code"] ✓

      "catsanddog":
        "cat" + "sanddog"
          "sand" + "dog" → ["cat","sand","dog"] ✓
        "cats" + "anddog"
          "and" + "dog" → ["cats","and","dog"] ✓
```

Solution Approach

Concept: Recursively check prefixes.

Algorithm

```
Algorithm WORD_BREAK(s, wordDict):
           result = []
           breakWord(0, [], s, wordDict, result)
           Return result

      Algorithm breakWord(start, current, s, wordDict, result):
           If start == n:
               result.add(join(current))
               Return
           For end = start+1 to n:
               substr = s.substring(start, end)
               If wordDict contains substr:
                   current.add(substr)
                   breakWord(end, current, s, wordDict, result)
                   current.remove(current.size()-1)
```

Java Implementation

```
class Solution {
           public List<String> wordBreak(String s, List<String> wordDict) {
               Set<String> dict = new HashSet<>(wordDict);
               List<String> result = new ArrayList<>();
               breakWord(0, new ArrayList<>(), s, dict, result);
               return result;
           }

           private void breakWord(int start, List<String> current, String s,
                                  Set<String> dict, List<String> result) {
               if (start == s.length()) {
                   result.add(String.join(" ", current));
                   return;
               }
               for (int end = start + 1; end <= s.length(); end++) {
```

```
                String word = s.substring(start, end);
                if (dict.contains(word)) {
                    current.add(word);
                    breakWord(end, current, s, dict, result);
                    current.remove(current.size() - 1);
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(2^n) | Space Complexity: O(n)

148. M Coloring Problem

Understanding the Problem NEW

Given a graph and a number of available colors m, determine if you can color every vertex such that no two directly connected vertices share the same color (and if so, find one valid coloring).

Brute Force Approach NEW

Trying every possible assignment of m colors to every vertex (m^V total combinations for V vertices) and checking validity only after a complete assignment is enormously wasteful, since most combinations will violate a constraint almost immediately.

Why We Optimize NEW

Backtracking again: assign a color to one vertex at a time, but only ever choose a color that doesn't conflict with any *already-colored neighbour* — checking this immediately rather than waiting until the whole graph is colored. If a vertex has no valid color available given the current partial coloring, backtrack and try a different color for an earlier vertex. This prunes away entire branches of invalid colorings the moment a conflict appears, rather than fully coloring the graph and discovering the conflict only at the very end.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Color graph with at most m colors (adjacent vertices different).

Visual Diagram

```
Graph: 4 vertices, edges: (0-1), (0-2), (1-2), (1-3), (2-3)

       Can color with 3 colors?
       0: Red
       1: Blue
       2: Green
       3: Red

       Check:
       0-1: Red-Blue ✓
```

```
       0-2: Red-Green ✓
       1-2: Blue-Green ✓
       1-3: Blue-Red ✓
       2-3: Green-Red ✓

       Backtracking:
       Color vertex 0: try color 0
       Color vertex 1: try color 1
       Color vertex 2: try color 2
       Color vertex 3: try color 0 ✓
```

Solution Approach

Concept: Assign colors vertex by vertex.

Algorithm

```
Algorithm M_COLORING(graph, m):
           colors = new int[n]
           If solve(0, graph, colors, m):
               Return colors
           Return []

       Algorithm solve(vertex, graph, colors, m):
           If vertex == n: Return true
           For color = 0 to m-1:
               If isSafe(vertex, color, graph, colors):
                   colors[vertex] = color
                   If solve(vertex+1, graph, colors, m):
                        Return true
                   colors[vertex] = -1
           Return false

       Algorithm isSafe(vertex, color, graph, colors):
           For neighbor in graph[vertex]:
               If colors[neighbor] == color:
                   Return false
           Return true
```

Java Implementation

```
class Solution {
           public int[] graphColoring(int[][] graph, int m) {
               int n = graph.length;
               int[] colors = new int[n];

              Arrays.fill(colors, -1);

              if (solve(0, graph, colors, m)) {
                  return colors;
```

```
            }
            return new int[0];
        }

        private boolean solve(int vertex, int[][] graph, int[] colors, int m) {
            if (vertex == colors.length) return true;

            for (int color = 0; color < m; color++) {
                if (isSafe(vertex, color, graph, colors)) {
                    colors[vertex] = color;
                    if (solve(vertex + 1, graph, colors, m)) {
                        return true;
                    }
                    colors[vertex] = -1;
                }
            }
            return false;
        }

        private boolean isSafe(int vertex, int color, int[][] graph, int[]
colors) {
            for (int neighbor : graph[vertex]) {
                if (colors[neighbor] == color) {
                    return false;
                }
            }
            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(m^n) | Space Complexity: O(n)

149. Sudoku Solver

Understanding the Problem NEW

Solve a partially-filled Sudoku puzzle by filling in the remaining cells so every row, column, and 3×3 box contains each digit 1-9 exactly once.

Brute Force Approach NEW

Trying every possible digit in every empty cell independently (9^(number of empty cells) total combinations) and validating the entire board only at the end is computationally infeasible for real puzzles.

Why We Optimize NEW

Backtracking with immediate validation: for each empty cell, try digits 1-9 one at a time, but only proceed with a digit if it doesn't *immediately* violate the row/column/box constraints given what's already filled in — if it's valid, recursively try to fill in the rest of the board; if that recursive attempt fails (some later cell has no valid options), backtrack, undo the current cell's digit, and try the next candidate digit instead. This immediate, per-cell validation (rather than filling the whole board blindly and checking once at the end) is what makes backtracking Sudoku solvers tractable even though the raw search space is enormous.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Solve a Sudoku puzzle.

Visual Diagram

```
Input:
       5 3 . | . 7 . | . . .
       6 . . | 1 9 5 | . . .
       . 9 8 | . . . | . 6 .
       ------+-------+------
       8 . . | . 6 . | . . 3
       4 . . | 8 . 3 | . . 1
       7 . . | . 2 . | . . 6
       ------+-------+------
       . 6 . | . . . | 2 8 .
       . . . | 4 1 9 | . . 5
```

```
. . . | . 8 . | . 7 9

Solution after backtracking:
5 3 4 | 6 7 8 | 9 1 2
6 7 2 | 1 9 5 | 3 4 8
1 9 8 | 3 4 2 | 5 6 7
------+-------+------
8 5 9 | 7 6 1 | 4 2 3
4 2 6 | 8 5 3 | 7 9 1
7 1 3 | 9 2 4 | 8 5 6
------+-------+------
9 6 1 | 5 3 7 | 2 8 4
2 8 7 | 4 1 9 | 6 3 5
3 4 5 | 2 8 6 | 1 7 9
```

Solution Approach

Concept: Place numbers and validate.

Algorithm

```
Algorithm SUDOKU_SOLVER(board):
           solve(board)
           Return board

       Algorithm solve(board):
           For each empty cell:
               For num = 1 to 9:
                   If isValid(board, row, col, num):
                       board[row][col] = num
                       If solve(board): Return true
                       board[row][col] = 0
               Return false
           Return true

       Algorithm isValid(board, row, col, num):
           Check row, column, and 3x3 box
```

Java Implementation

```
class Solution {
           public void solveSudoku(char[][] board) {
               solve(board);
           }

           private boolean solve(char[][] board) {
               for (int row = 0; row < 9; row++) {
                   for (int col = 0; col < 9; col++) {
                       if (board[row][col] == '.') {
                            for (char num = '1'; num <= '9'; num++) {
```

```
                            if (isValid(board, row, col, num)) {
                                board[row][col] = num;
                                if (solve(board)) return true;
                                board[row][col] = '.';
                            }
                        }
                        return false;
                    }
                }
            }
            return true;
        }

        private boolean isValid(char[][] board, int row, int col, char num) {
            // Check row
            for (int i = 0; i < 9; i++) {
                if (board[row][i] == num) return false;
            }

            // Check column
            for (int i = 0; i < 9; i++) {
                if (board[i][col] == num) return false;
            }

            // Check 3x3 box
            int boxRow = (row / 3) * 3;
            int boxCol = (col / 3) * 3;
            for (int i = boxRow; i < boxRow + 3; i++) {
                for (int j = boxCol; j < boxCol + 3; j++) {
                    if (board[i][j] == num) return false;
                }
            }

            return true;
        }
    }
```

Time & Space Complexity

Time Complexity: O(9^(n²)) | Space Complexity: O(n²)

150. Expression Add Operators

Understanding the Problem NEW

Given a string of digits and a target value, insert the operators +, -, and × between the digits (without rearranging them) so the resulting expression evaluates to the target. Find every way to do this.

Brute Force Approach NEW

Since there are 3 possible operators to insert at each of the gaps between digits, generating every possible operator arrangement means the search space grows exponentially (3^(number of gaps)) — but there's no way around exploring this space in general, since almost any subset of insertions could be the right one.

Why We Optimize NEW

The main practical technique here (beyond plain exhaustive recursion trying each operator at each gap) is careful handling of **multi-digit numbers** and **multiplication's precedence**: at each recursive step, you must also consider grouping multiple consecutive digits together as one multi-digit number (not just single digits), and when evaluating running totals, you need to track the *previous operand* separately so that a multiplication can correctly 'undo and redo' the last addition/subtraction it affects (since × binds tighter than + or -). Getting this precedence bookkeeping right, layered on top of the exhaustive recursive search over operator choices, is what separates a working solution here from a subtly incorrect one — this is less about complexity optimization and more about correctly modeling arithmetic precedence within a backtracking search.

ORIGINAL CONTENT CONTINUES BELOW (UNCHANGED)

Add operators to get target value.

Visual Diagram

```
Input: "123", Target: 6
       Expressions:
       1+2+3 = 6 ✓
       1*2*3 = 6 ✓
```

```
Input: "232", Target: 8
2+3*2 = 8 ✓
2*3+2 = 8 ✓

DFS:
"123" at position 0:
  Take "1":
     expr = "1", val = 1, prev = 1
     at pos 1:
       Take "2":
         +: expr="1+2", val=3, prev=2
         -: expr="1-2", val=-1, prev=-2
         *: expr="1*2", val=2, prev=2
```

Solution Approach

Concept: DFS with operator choices.

Algorithm

```
Algorithm ADD_OPERATORS(num, target):
           result = []
           dfs(0, "", 0, 0, num, target, result)
           Return result

       Algorithm dfs(index, expr, val, prev, num, target, result):
           If index == n:
               If val == target: result.add(expr)
               Return
           For i = index to n-1:
               curr_str = num.substring(index, i+1)
               curr = Integer(curr_str)
               If index == 0:
                   dfs(i+1, curr_str, curr, curr, num, target, result)
               Else:
                   dfs(i+1, expr+"+"+curr_str, val+curr, curr, ...)
                   dfs(i+1, expr+"-"+curr_str, val-curr, -curr, ...)
                   dfs(i+1, expr+"*"+curr_str, val-prev+prev*curr, prev*curr, ...)
               If curr == 0: Break
```

Java Implementation

```
class Solution {
           public List<String> addOperators(String num, int target) {
               List<String> result = new ArrayList<>();
               dfs(0, "", 0, 0, num, target, result);
               return result;
           }

           private void dfs(int index, String expr, long val, long prev,
```

```
                        String num, int target, List<String> result) {
            if (index == num.length()) {
                if (val == target) {
                    result.add(expr);
                }
                return;
            }

            for (int i = index; i < num.length(); i++) {
                // Skip numbers with leading zeros
                if (i > index && num.charAt(index) == '0') break;

                String currStr = num.substring(index, i + 1);
                long curr = Long.parseLong(currStr);

                if (index == 0) {
                    dfs(i + 1, currStr, curr, curr, num, target, result);
                } else {
                    dfs(i + 1, expr + "+" + currStr, val + curr, curr, num,
target, result);
                    dfs(i + 1, expr + "-" + currStr, val - curr, -curr, num,
target, result);
                    dfs(i + 1, expr + "*" + currStr, val - prev + prev * curr,
prev * curr, num, ta
                }
            }
        }
    }
```

Time & Space Complexity

Time Complexity: O(4^n) | Space Complexity: O(n)

Recursion Summary

Problem	Type	Time	Space	Key Technique
Atoi	Easy	O(n)	O(n)	State tracking
Pow(x,n)	Easy	O(log n)	O(log n)	Fast exponentiation
Good Numbers	Easy	O(log n)	O(1)	Modular arithmetic
Sort Stack	Medium	$O(n^2)$	O(n)	Insert sorted
Reverse Stack	Medium	$O(n^2)$	O(n)	Insert at bottom
Binary Strings	Subsequence	$O(2^n)$	$O(2^n*n)$	Include/Exclude
Generate Parentheses	Subsequence	$O(4^n/\sqrt{n})$	$O(4^n/\sqrt{n})$	Valid combinations
Subsequences	Subsequence	$O(2^n)$	$O(2^n*n)$	Include/Exclude
Subsequence Patterns	Subsequence	$O(2^n)$	O(n)	Print/Count/Check
Count Sum K	Subsequence	$O(2^n)$	O(n)	Counting
Exists Sum K	Subsequence	$O(2^n)$	O(n)	Early termination
Combination Sum	Subsequence	O(2^(target/min))	O(target/min)	Unlimited elements
Combination Sum II	Subsequence	$O(2^n)$	O(n)	Unique, skip duplicates
Subset Sum I	Subsequence	$O(2^n)$	$O(2^n)$	Sum calculation
Subset Sum II	Subsequence	$O(2^n)$	$O(2^n*n)$	Unique subsets
Combination Sum III	Subsequence	O(9!/(9-k)!)	O(k)	Fixed length
Letter Combinations	Subsequence	$O(4^n)$	$O(4^n)$	Mapping
Palindrome Partitioning	Hard	$O(n*2^n)$	O(n)	Check palindrome
Word Search	Hard	$O(m*n*4_3)$	O(L)	DFS with backtracking
N Queen	Hard	O(n!)	$O(n^2)$	Column validation
Rat in Maze	Hard	$O(4^{(n^2)})$	$O(n^2)$	Direction exploration
Word Break	Hard	$O(2^n)$	O(n)	Prefix matching
M Coloring	Hard	$O(m^n)$	O(n)	Vertex coloring
Sudoku Solver	Hard	$O(9^{(n^2)})$	$O(n^2)$	Cell validation

Expression Operators	Hard	$O(4^n)$	O(n)	Operator combinations

— End of enhanced Chapter 6: Recursion — remaining chapters continue in the full edition —